代码生成越来越快，软件交付真正的瓶颈转向了哪里？

机器之心 阿里技术 2026年9月24日 03:02 浙江 21人



这是2026年的第 63 篇文章

（ 本文阅读时间：约 10 分钟 ）

2026 年 9 月 22 日，阿里巴巴集团 CEO 吴泳铭在杭州云栖大会发表演讲，阐述对「机器智能」时代的思考：机器正在成为思考的主力，智能正在成为一种规模化的商品，未来机器思考总量将达到人类的 1000 倍以上 —— 正如上一次工业革命把「动力」变成了规模化的商品。

与技术底座演进同步，阿里技术人的研发方式也在加速向 AI Native 转变：Agent 不再只用于生成代码，而是逐步进入需求、开发、测试、发布和运维各环节。

9 月 23 日，「智启原生 研见未来」阿里巴巴集团 AI Native 研发实践论坛在云栖大会举办。

机器之心联合创始人、主编李亚洲，阿里巴巴研发基础设施负责人许晓斌，千问事业部杰出技术专家李英各，蚂蚁集团平台技术事业群总架构师黄挺，飞猪 CTO 陈烨，以及复旦大学计算与智能创新学院副院长彭鑫，分别从内容生产、业务研发、基础设施和软件工程研究出发，与现场技术人交流各自的 AI Native 实践。

这些实践共同指向一个更靠后的工程问题：当代码生成越来越快，软件交付真正的瓶颈转向了哪里？

> 注：文中效率、规模和业务指标均来自嘉宾现场分享与演示材料。各团队统计对象和口径不同，不宜直接横向比较。

许晓斌：

Agent 进入生产，需要一套完整的工程系统

阿里巴巴研发基础设施负责人许晓斌从内部实践切入，讨论 Agent 如何进入真实生产链路。首先遇到的不是代码生成问题，而是它在哪里运行、如何验证结果。阿里开源的 OpenSandbox，在内部每周创建量已经超过千万级，服务于 Agent 运行、训练和评测。另一套环境体系已经支持约两三千个应用在独立环境中完整复原，并计划很快扩展到一万个应用。环境不再只是开发前的准备工作，而是 Agent 能否持续执行任务的基础设施。

运行环境逐渐成熟后，生产操作也开始交给 Agent。Guardrail 已经在几十个应用、约百人规模的团队中用于灰度发布和运维，目标是在半年内让一半生产系统的发布由 Agent 自主完成，人只负责最终确认。人工并没有退出链路，而是留在授权和高风险决策环节；信息收集、证据核对和标准化执行则更多交给系统。



图 / 阿里巴巴研发基础设施负责人许晓斌

为什么编码提速没有直接转化为交付提速？许晓斌给出了一组时间结构：在阿里巴巴，编码通常只占软件交付总时长的 20%—30%。他还展示了一个用于说明问题的极端链路：编码不足整体工作的 1%，大约只需 1 小时，但从需求进入到最终交付仍要约 21 天，时间主要耗在影响分析、环境准备、联调、审批、灰度和封网等环节。这个案例并非真实项目数据，却清楚展示了局部提速和整体提速之间的差距。

沿着完整交付链路，许晓斌将 AI Native 研发拆成需求、上下文、环境、身份和证据几个相互关联的部分。人类工程师可以从会议、聊天记录和经验中补齐隐含信息，Agent 没有这些默认知识。需求必须从一句模糊描述变成完整、精确、能够运行和验证的任务表达；需求来源也要被保留下来，使后续代码变更能够与最初意图进行比对。

他把这套组织方式概括为「一个需求，一组事实」。产品文档、技术方案、设计稿、前后端工程、测试用例和验证证据不再散落在不同系统里，而是围绕同一个需求形成连续上下文。代码仓库只是软件事实的一部分，配置、依赖、运行环境、数据基线和验收标准同样需要被 Agent 读取。

需求被说明白之后，任务还要真正跑得起来。Agent 需要知道工具链如何安装、依赖服务在哪里、网络边界如何设置、运行结果怎样反馈。Sandbox 提供一个可执行、受约束、可以丢弃的空间；应用环境复原则让 Agent 能够在接近真实系统的条件下完成开发和验证。这也解释了 OpenSandbox 和应用环境体系为何会成为 Agent 规模化落地的前提。

当 Agent 从开发环境走向生产，系统还必须回答它代表谁、可以做什么。许晓斌将 User Identity、Agent Identity 和 Workload Identity 组合成复合身份，用于授权、策略判断、审计和高危操作再确认。一次操作不仅要关联发起它的 Agent，还要关联背后的用户、任务目标和实际运行负载。



图 / 论坛现场

李英各：

用统一上下文和验证机制降低交付成本

千问事业部杰出技术专家李英各关注的是个人编码效率与团队交付效率之间的落差。AI 编码效率大幅提高，并没有带来同等幅度的整体交付提速。长链路、跨角色协同造成的上下文不完整，以及上线前繁重的验收工作，仍在消耗大量成本。

针对这一问题，李英各提出 Context & Verifier Driven Developer。产品、设计、研发和测试在同一工作台上工作，通过需求评测 Agent 辅助提升 PRD 质量；研发负责架构和方案约束，并作为代码质量的第一责任人，与测试共同完善编码 Loop 里的验证机制。统一上下文、编码

Loop 和上线前验收被连成一个大循环，以此降低验收成本，释放 AI 在软件工程全链路的价值。



图 / 千问事业部杰出技术专家 李英各

千问团队把实践中的问题概括为「一个发现、一个高估、两个低估」：个人效率不会自动转化为团队吞吐；团队容易高估编码提速和自由生成代码的质量，同时低估跨角色沟通、返工成本和验证工作的复杂度。

在组织方式上，千问正在让产品、设计、研发和测试围绕同一个目标、同一份上下文和同一组质量证据工作，并通过更内聚的虚拟团队缩短跨角色链路。个人使用什么工具不再是唯一问题，信息能否在团队中连续流动、结果能否被共同验证，开始决定整体吞吐。

黄挺：

当基础设施的用户从工程师变成 Agent

蚂蚁集团平台技术事业群总架构师黄挺带来的实践，落在 Agent 参与线上问题修复之后的基础设施变化。蚂蚁把问题分派、云上 Agent、隔离评测和人工发布确认串成一条 Badcase 修复链路。以「阿福」 为例，单个 Badcase 平均修复耗时降低到原来的四分之一，人工投入从天级别降至少于一小时。

支撑这条链路的 ARCA 隔离沙箱，现场披露的指标包括 200 毫秒容器冷启动、单沙箱 8MB 内存开销、单集群每秒 800 次创建吞吐和日均 60 万次以上的创建量。



图 / 蚂蚁集团平台技术事业群总架构师黄挺

黄挺将这种变化概括为：「基础设施产品的用户从工程师变成了 Agent。」过去，研发平台默认操作者是人。人会阅读提示，在高风险操作前停下来，也能感知一次变更可能带来的影响；Agent 则会围绕目标连续调用工具、修改文件和触发流程。原先依赖人的经验和谨慎维持的边界，需要转化为标准接口、权限分层、隔离环境、限流和审计能力。

陈烨：

把 AI 放到需求入口，重新设计研发流程

飞猪 CTO 陈烨从业务研发链路出发，把 AI 放到需求入口。飞猪通过领域知识库和 PRD Gate 检查需求是否自洽、会影响哪些系统、需要哪些资源，再把后续研发流程线上化、标准化。平台技术团队一个需求从评审到上线的端到端平均周期，由 2—4 周缩短至 1 周。过去一年，平台型技术有超过三成的业务需求由 AI 流程支撑，住宿业务有超 60% 的应用已接入 AI 研发流程，商旅业务的组件生产效率也提升超 70%。

陈烨用阿姆达尔定律解释编码效率与整体效率之间的差距。假设写代码占全部工作的一半，即使 AI 把编码速度提高 100 倍，整体效率也只会接近 2 倍。局部环节越快，原本不显眼的等待、沟通和串行流程越容易成为新瓶颈。



图：飞猪 CTO 陈烨

在陈烨的划分中，AI 研发经历三个阶段：Copilot 阶段，AI 是辅助工具；Agent 阶段，AI 开始主导执行；到了 AI Native 阶段，Agent 成为团队成员，流程本身需要围绕它重新设计。人的位置并没有消失，而是向任务定义、结果判断和风险决策移动。

陈烨将代码生成提速后真正成为瓶颈的事归纳为三件：搞清楚要做什么、判断结果对不对、把知识写下来。他的结论是：「AI Native 不是一个技术选择，是一个组织选择。」

彭鑫：
用「可驾驭性」判断 AI 的作用边界

复旦大学计算与智能创新学院副院长彭鑫把讨论拉回软件工程研究。组织重构并不意味着所有任务都适合交给 AI。一次性方案、流程明确的工具应用和需要长期运行的复杂系统，对 AI 提出的要求并不相同。



图 / 复旦大学计算与智能创新学院副院长 彭鑫

彭鑫用「可驾驭性」判断 AI 的作用边界：任务能否清楚定义，结果能否客观验证，环境能否被 Agent 理解、导航和操控。面对复杂系统，还要持续记录需求、架构、决策和演化历史，形成与代码对应的「数字孪生」。

李亚洲：

个人效率提升之后，工作流还要继续演进

机器之心联合创始人、主编李亚洲站在非研发的视角，从内容生产场景提供了另一组观察。机器之心编辑部过去报道一篇论文，通常需要两名编辑用一上午完成翻译、整合、校对、排版和发布；现在，一名编辑借助 Agent，一个多小时即可完成。据他现场估计，效率提高后，编辑部承接的工作量也增至此前约三倍。



图 / 机器之心联合创始人 & 主编 李亚洲

与此同时，不同编辑仍在使用不同工具和 Prompt。个人产出已经提高，统一工作流却还没有形成。这个案例虽然不属于软件开发，却同样说明个人效率不会自然转化为稳定的团队流程。

从六组实践看 AI Native 研发的共同方向

六位嘉宾分别从内容生产、需求管理、团队协作、基础设施、生产治理和软件工程研究出发，呈现了 Agent 从辅助编码走向真实生产的同一条路径。随着它进入需求、开发、测试、发布和运维，研发关注点也从生成速度移向可验证交付。上下文、测试、沙箱、身份和权限成为 Agent 进入生产必须具备的工程条件，团队也要重新安排人的判断与 Agent 的执行。

论坛现场，《AI Native 研发范式实践手册》正式发布。这份手册由阿里巴巴十余名工程师及不同角色的实践者共同贡献，将基础设施、开源项目和业务一线的经验汇集为可复用的方法，也为现场讨论提供了进一步落地的载体。



回到论坛提出的问题，软件没有随着代码生成同步加速，症结落在完整交付链路上。需求是否清楚、上下文是否连续、环境能否复现、结果能否验证、生产操作是否可治理，共同决定最终速度。

六位嘉宾从不同方面给出的实践，共同呈现出 AI Native 研发范式正在形成的方向：技术系统、交付流程和组织方式需要同步变化，Agent 才能真正进入生产。

*附：手册下载地址

https://ai-native.alistatic.com/app/ainativeinfra/ai-native-handbook-web/index（复制链接到浏览器打开即可下载）



欢迎留言一起参与讨论~

AI Native · 目录

<上一篇
AI 创新实践丨「架构师 Agent」系统化落地

下一篇 >
从榜单走向真实业务负载：Kernel Agent 刷新 Qwen3.8-Max 核心推理算子性能

阅读 3282

留言

写留言

还没有留言，来写首评